

Worldline Round 2 SDET / Test Automation Lead

Topic-Wise Preparation — Part 2 (Complete)

COMPLETE RUNNING RECORD

This edition preserves the earlier Part 2 material and adds the later debugging, test-data, contract, response-validation and security questions. The overlapping Q34/Q35 numbering is also preserved so no covered scenario is lost.

Focus: API reliability, debugging, concurrency, mocking, idempotency, contracts, validation and security.

Interview: Worldline Global — Round 2

Index / Complete Question Map

Section	Coverage
A	API Failure & Reliability — Q32–Q35
B	Debugging & Failure Analysis — Q34–Q39
C	API Contracts & Lifecycle — Q40–Q43
D	Response Validation & Security — Q44–Q45
E	Earlier Part 2 Core Material — preserved
F	Last-Minute Revision Sheet

NUMBERING NOTE

The preparation conversation had an overlap around Q34/Q35. Both versions are included: Q34 Gateway Processed Payment but Response Failed, Q34 Debugging a Failed API Test, Q35 API Returns 200 but DB PENDING, and Q35 Automation Failure vs Application Failure.

A. API Failure & Reliability

Q32 — API Failure Handling: Payment Gateway Returns HTTP 500

Scenario: A payment API calls a third-party gateway and the gateway returns HTTP 500.

User's core answer: "Make sure payment not succeeded, proper error handling, idempotency, timeout etc."

Senior additions

- Do not mark payment successful merely because an error was received.
- Validate error status, error code/message, content type and safe response.
- Check retry count/backoff and timeout handling.
- Use idempotency/transaction identity to prevent duplicate payments.
- Verify payment/order/database state consistency.
- Use correlation IDs and safe logs; do not expose sensitive data.

INTERVIEW-READY

"I would validate the business impact, not just the HTTP status. I would verify payment state, retry/timeout behaviour, idempotency, database consistency and correlation logging."

Q33 — Retry After 500: Avoid Duplicate Payment

1. First request -> gateway returns 500
2. Application retries
3. Same idempotency key / transaction identity is reused
4. Verify only ONE payment transaction/charge exists
5. Verify final state is consistent
6. Verify retry count and backoff

KEY TRAP

The first request may have been processed even though the response failed. Retrying with a new transaction can double-charge the customer.

Q34 — Gateway Processed Payment but Application Received 500/Timeout

Critical: Timeout/500 does not necessarily mean payment failed; the outcome may be unknown.

- Simulate gateway processing the payment but returning timeout/500.

- Do not blindly create a new payment.
- Retry with the same idempotency key where supported.
- Use gateway status API, webhook/callback or reconciliation to determine outcome.
- Verify eventual application state and exactly one successful payment.
- Correlate gateway/application logs.

STRONG LINE

"I would never equate a timeout or 500 response with definite payment failure. The outcome can be unknown, so I would reconcile before allowing another charge."

Q35 — API Returns 200 SUCCESS but Database Still Shows PENDING

- First determine whether processing is synchronous or asynchronous.
- For async processing, PENDING may be expected; validate eventual SUCCESS within SLA.
- For sync processing, investigate transaction/DB consistency.
- Verify transaction ID, amount, state transition, duplicates, logs and downstream processing.

INTERVIEW-READY

"I would first determine whether PENDING is an expected intermediate state. If asynchronous, I validate eventual consistency within SLA; if synchronous, I investigate a possible consistency defect."

B. Debugging & Failure Analysis

Q34 — Debugging a Failed API Test

Scenario: Expected 200, actual 500, after 20 successful executions.

- Verify URL, method, headers, authentication and payload.
- Inspect response body/error details.
- Check correlation ID and application logs.
- Check deployment/configuration/database/downstream changes.
- Reproduce manually using the same request/environment.
- Verify test data and interference from other tests/processes.
- Check timing, concurrency, async processing and environment resources.
- Classify as product, automation, environment/dependency or data issue.

SENIOR THINKING

"Because it was stable for 20 runs, I would investigate environment, dependency, deployment, data and intermittent behaviour before changing the automation."

Q35 — Automation Failure vs Application Failure

Scenario: Automation gets 500; the same request in Postman in the same environment also gets 500.

- Prove request equivalence: URL, method, headers, auth, parameters and body.
- Inspect response and correlation ID.
- Check application/downstream logs and recent changes.
- If equivalent and Postman also fails, investigate application/environment/dependency rather than assuming automation is broken.

STRONG LINE

"If the same request fails in Postman too, I would prove equivalence and then investigate the application, environment or dependency."

Q36 — Product Defect or Test-Data Problem?

Scenario: A100 sends 500 and API returns 400 Insufficient Balance.

- Verify actual balance through authoritative DB/API.
- Check whether another test/process changed it.
- Check shared mutable data.
- Reset/create controlled data where appropriate.
- Run with known-valid data.
- If sufficient balance exists and rejection remains, investigate product defect.

KEY PRINCIPLE

A failed test does not automatically mean a product defect. Validate preconditions and data first.

Q37 — Parallel Execution Using the Same Account

Scenario: 10 tests share A100 balance 10,000; each withdraws 1,000; sometimes 8/9/10 succeed.

- Possible test-data isolation problem: tests change each other's state.
- Possible application race condition: concurrent updates are unsafe.
- Compare sequential vs parallel execution.
- Normal functional tests should isolate mutable data.
- Concurrency tests should share intentionally and verify safe behaviour.

EXPECTED THINKING

The application must never permit an invalid negative balance or unsafe lost update. The exact number of successes depends on the business concurrency rules.

Q38 — Intermittent API Failure

Scenario: 100 tests normally pass; occasionally 98 pass and 2 fail; individual reruns pass.

- Capture request/response, timestamp, data and transaction/correlation ID.
- Check application/dependency logs.
- Compare sequential vs parallel execution.
- Investigate shared data, environment instability, downstream failures, races and async timing.
- Check recent deployment/configuration.
- Use retry only as controlled diagnosis or actual product behaviour—not to hide defects.

KEY LINE

"Intermittent failures need evidence and pattern analysis, not simply a rerun."

Q39 — Retry or Defect?

Scenario: API returns 503, retry returns 200, so the framework passes.

- Determine whether 503 is expected transient behaviour.

- If retry is designed, validate retry count, backoff, final business state and idempotency.
- Record the initial 503 rather than hiding it.
- If retry is not expected, treat the initial 503 as a failure/reliability signal.
- Do not blindly retry 400/401/403.

MEMORISE

"Retry is a resilience mechanism, not a failure-hiding mechanism."

C. API Contracts & Lifecycle

Q40 — API Contract Change

Scenario: Existing response has transactionId/status/amount(number); new response adds currency.

- Passing tests do not automatically prove compatibility.
- Determine whether currency is optional or mandatory.
- Check schema/documentation and existing consumers.
- Check strict response comparisons/schema validation/generated models.
- If breaking, version or use controlled rollout.

SENIOR THINKING

"I would assess backward compatibility, not just rely on existing tests passing."

Q41 — API Versioning

Scenario: v1 amount is a number; v2 amount is an object containing value + currency. Breaking change.

- Keep v1 unchanged during migration.
- Introduce v2 with the new contract.
- Test v1 regression/contract compatibility.
- Test v2 positive/negative/boundary/error scenarios.
- Use consumer/contract tests where appropriate.
- Plan consumer migration and validate business outcomes.

INTERVIEW LINE

"For a breaking contract change, I would introduce a new API version rather than silently changing the existing contract."

Q42 — API Deprecation

Scenario: v1 is in production, v2 exists, but mobile clients still call v1. Development wants to remove v1.

- Identify active consumers via API gateway/access logs, metrics and consumer IDs where available.
- Coordinate migration of remaining consumers.
- Maintain regression/contract coverage.
- Monitor v1 usage.
- Decommission only after monitoring confirms no active consumers and the approved migration/deprecation plan is complete.

WHAT DOES ZERO USAGE MEAN?

No active applications/clients are making calls to v1. Example: usage falls from 7,500 calls to 0 after migration.

Q43 — API Contract Testing

Scenario: Provider changes amount from number to string but still returns HTTP 200.

- Validate JSON schema: names, types, required fields and allowed values.
- Use consumer-driven contract testing where appropriate.
- Pact can store consumer expectations and let the provider verify them.
- Run contract checks in CI/CD to catch breaking changes before production.

SCHEMA VS CONTRACT

Schema validation asks: "Is the response structurally valid?" Contract testing asks: "Does the provider still satisfy what this consumer expects?"

D. API Response Validation & Security

Q44 — API Response Validation

```
{
  "transactionId": "TX123",
  "status": "SUCCESS",
  "amount": 1000,
  "currency": "INR"
}
```

- HTTP status code.
- Response body and business values.
- JSON schema, types and required/optional fields.
- Headers such as Content-Type.
- Authentication and authorisation behaviour.
- Response-time SLA.
- Actual payment/order/database state.
- Negative cases: invalid ID, missing fields, invalid amount, invalid auth, duplicate requests.

MOST IMPORTANT TRAP

HTTP 200 does not equal PASS. Validate business outcome and response contract too.

Q45 — API Security Testing

```
POST /payment
Authorization: Bearer <valid-token>
```

```
{
  "accountId": "A100",
  "amount": 1000
}
```

1. Authentication — Who are you?

- Valid token -> success.
- Missing token -> 401.
- Invalid token -> 401.

- Expired/revoked/malformed token -> 401.

2. Authorisation — Are you allowed?

Authenticate as a user who owns A100, then try operating on another user's B200. Expected result is typically 403 Forbidden.

MEMORISE

401 = authentication/identity problem. 403 = authenticated, but not authorised.

3. Input Manipulation

- Negative amount.
- Zero amount.
- Extremely large amount.
- Wrong type such as string instead of number.
- Null/missing fields.
- Unexpected fields.
- Malformed or malicious input.

4. Sensitive Data Exposure

- Check responses, errors and logs for tokens, credentials, payment details, stack traces or internal infrastructure details.
- Verify sensitive values are protected/masked and only required information is returned.

5. API Abuse / Rate Limiting

- Excessive requests.
- Throttling/rate limiting.
- Appropriate 429 behaviour where designed.
- Service stability.
- Repeated payment requests must not create duplicate charges.

SENIOR INTERVIEW ANSWER

"I would cover authentication, authorisation, input validation, sensitive-data exposure and API abuse. I would test missing/invalid/expired tokens; verify users cannot access another user's account; test malformed and boundary inputs; check that sensitive data is not exposed; and validate rate limiting and idempotency for repeated payments. I would validate both HTTP response and actual business impact."

E. Earlier Part 2 Core Material — Preserved

This material was already part of the earlier Part 2 preparation and is intentionally retained.

Third-Party API Mocking

- Do not make every test depend on a live third party.
- Simulate 200 success, 400/402/declined, 500, timeout/no response, connection failure and malformed responses.
- Mockito is primarily for Java dependencies; WireMock/MockServer/Postman Mock Server are useful for HTTP simulation.

```
when(paymentClient.makePayment(1000)).thenReturn("SUCCESS");  
verify(paymentClient).makePayment(1000);
```

Stubbing: when(...).thenReturn(...) defines a mock response. **Verification:** verify(...) checks the interaction.

HONEST PROJECT WORDING

"In my previous project, the mock server was already set up for the required third-party APIs. My role was primarily to leverage those mocked endpoints in automation tests rather than build and maintain the mock-server infrastructure myself. I used predefined responses to validate different API scenarios."

Idempotency Quick Recall

Method	Typical behaviour
GET	Idempotent
PUT	Idempotent
DELETE	Idempotent in intended state effect
POST	Generally non-idempotent
PATCH	Depends on semantics

POST /payment

Idempotency-Key: ABC123

Same request + same key repeated:

-> return original outcome

-> do not create another charge

Synchronous vs Asynchronous

- Synchronous: request waits for processing and response normally represents completion.
- Asynchronous: request may return PENDING/accepted while processing continues.
- Testing must respect designed state transitions and SLA.

F. Last-Minute Revision Sheet

1. A failed test does not automatically mean a product defect.
2. Validate preconditions and test data before changing automation.
3. A timeout or HTTP 500 does not necessarily mean payment failed.
4. Retry is a resilience mechanism, not a failure-hiding mechanism.
5. Idempotency prevents repeated payment requests from creating duplicate business effects.
6. PUT/DELETE are idempotent in intended state effect; POST is generally not.
7. PATCH can be idempotent or non-idempotent depending on semantics.
8. Authentication = who you are; authorisation = what you are allowed to do.
9. HTTP 200 does not equal a valid business response.
10. Contract testing protects the agreement between API consumers and providers.
11. Schema validation checks structure; contract testing checks consumer expectations.
12. Intermittent failures need evidence before reruns or automation changes.
13. Normal functional tests isolate mutable data; concurrency tests share intentionally.
14. Payment tests must verify business outcome, state consistency and duplicate prevention.
15. Senior SDET answers explain the reasoning and investigation path, not just the final result.

FINAL PREPARATION ADVICE

This scenario/API section is now broad enough. Move next to programming/coding, API framework architecture, Java, Selenium/Playwright and lead/managerial scenarios, then finish with a mock interview.

Coverage Assurance

Included: Q32, Q33, both Q34 variants, both Q35 variants, Q36, Q37, Q38, Q39, Q40, Q41, Q42, Q43, Q44 and Q45, plus the earlier Part 2 mocking, idempotency and reliability material.

The overlapping numbering is deliberately preserved rather than silently dropping either scenario.